

UNIT V

SYLLABUS

HBase – Data Model and Implementation
HBase Clients, Examples
Cassandra – Data Model, Cassandra Examples, Clients
Hadoop Integration
Pig – Data Model
Grunt Shell
Pig Latin – Developing
Testing Pig Latin Scripts
Hive – Data Types, File Formats
HiveQL Data Definition, Data Manipulation, Queries

HBase – Data Model and Implementation

HBase is the Hadoop database and is a distributed and scalable, column-oriented database similar to Google Big Table. HBase provides random, real-time access to data in the cluster. HBase is designed for hosting very large tables with billions of rows and millions of columns.

One of the unique qualities of Hbase is it doesn't care about data types because we can store different data types of data for the same column in different rows. It contains different sets of tables that maintain the data in key-value format. Hbase is best suitable for sparse data sets which are very common in the case of big data. It can be used to manage structured and semi-structured data and it has many built-in features such as:

- Scalability
- Versioning
- Compression
- Garbage collection

There are two types of data storage mediums:

1. Row-Oriented
2. Column-Oriented

Row-Oriented :

In the Row-Oriented data storage approach, the data is stored and retrieved one row at a time. This could lead to several problems, suppose we want only some part of the data from the row but according to this approach you have to retrieve the complete row even if you don't need it. Apart from that this approach also serves to get help in the case of the OLTP systems operation and it helps in easy to read and write the records. But it is less efficient in the case when we perform operations on a complete database.

Column-Oriented :

In the column-Oriented data storage approach, the data is stored and retrieved based on the columns. Thus the problem which we were facing in the case of the row-oriented approach has been solved because in the column-oriented approach we can filter out the data which are required to us from the whole set of data with the help of corresponding columns. In the column-oriented approach, the read and write operations are slower than others but it can be efficient while performing operations on the entire database and hence it permits very high compression rates.

Preference Of HBase :

- Hbase can handle large databases very easily as compare to other databases and perform the operations in an efficient manner.
- The data expected in Hbase are highly structured and can easily fit in the well-defined schema.
- It is easy and suitable for low latency operations.
- It provides access to the particular row from thousands of records.
- Data in Hbase can be accessed through shell commands or through client API in java.
- Other databases get extremely slow while large databases but in the case of Hbase it handles the large databases in a finite manner.

Advantages of HBase :

- Hbase provides great functionality for analytics in association with Hadoop MapReduce.

- It is capable of handling a very large amount of databases.
- Hbase provides the facility of sharing the database with other users.
- There are many operations like data reading and processing that will take less amount of time as compared to other traditional databases.
- Whenever there is a failure or load sharing problem arises Hbase can automatically recover from the problem because it is internally distributed.
- In Hbase scalability is supported in both linear and modular forms.
- It provides a large consistency.

Problems in HBase :

- In HBase the memory blocks requirements and hardware Requirement for allocation during an operation is expensive.
- To store the Large file which is in binary format is very difficult in HBase.
- There is no query optimizer present in HBase due to which it is unable to perform SQL functions and it also doesn't support SQL structures.
- Though there is a lot of functionality provided by Hbase it can't serve as a complete replacement of the traditional models because some of the traditional features are still not supported by HBase.
- Hbase does not provide any transaction support.
- HBase does not support any default indexing functionality.

HBase Clients

HBase provides different types of clients to interact with tables for read/write operations. These clients allow developers and users to perform CRUD operations programmatically or via the command line.

1. Java API (Native HBase Client)

- Most widely used client.
- Provides **low-level, high-performance** access.
- Uses classes like Connection, Table, Put, Get, Scan, Delete.
- Suitable for enterprise applications needing tight integration.

Example (Java – Put & Get)

```
Configuration config = HBaseConfiguration.create();
Connection conn = ConnectionFactory.createConnection(config);
Table table = conn.getTable(TableName.valueOf("students"));

Put put = new Put(Bytes.toBytes("101"));
put.addColumn(Bytes.toBytes("info"), Bytes.toBytes("name"),
Bytes.toBytes("Sam"));
table.put(put);

Get get = new Get(Bytes.toBytes("101"));
Result result = table.get(get);
System.out.println(Bytes.toString(result.getValue(Bytes.toBytes("info"),
Bytes.toBytes("name"))));
```

2. HBase Shell (Command-Line Client)

- Interactive command-line interface.
- Best for **administration**, **quick testing**, and **manual operations**.

Example Commands

```
hbase shell
create 'students', 'info'
put 'students', '101', 'info:name', 'Sam'
get 'students', '101'
scan 'students'
delete 'students', '101', 'info:name'
```

3. REST Client (Stargate)

- Provides **HTTP-based** access to HBase.
- Useful when non-Java clients need access (Python, JavaScript, etc.).

- Returns JSON/XML data.

Example (REST – cURL)

```
curl -X GET http://localhost:8080/students/101/info:name
```

4. Thrift Client

- Language-neutral RPC framework.
- Allows HBase to be used with **Python, PHP, Ruby, C++, Go**, etc.
- Lightweight and faster than REST.

Example (Python using Thrift)

```
from hbase import Hbase
```

```
client = Hbase.Client(protocol)
```

```
client.mutateRow("students", "101",  
    [Hbase.Mutation(column="info:name", value="Sam")])
```

5. MapReduce Client (Table Input/Output Format)

- Used for **batch operations** on HBase tables.
- Allows MapReduce jobs to read/write HBase directly.

Example

- Use TableInputFormat to read HBase table as input.
- Use TableOutputFormat to write results back into HBase.

6. Spark-HBase Connector

- Allows using **Apache Spark** to read/write HBase.
- Used in analytics pipelines.

Simple Example (Pseudocode)

```
val df = spark.read.format("hbase").option("table", "students").load()  
df.show()
```

Summary

Client Type	Use Case
Java API	High-performance applications
HBase Shell	Admin tasks/testing
REST API	Access from any HTTP-enabled client
Thrift API	Multi-language support
MapReduce	Batch processing
Spark Connector	Advanced analytics

Cassandra – Data Model, Cassandra Examples, Clients

Cassandra's data model is a partitioned row store with tunable consistency, optimized for fast writes and reads in distributed environments. It is a NoSQL wide-column store database.

- **Tables (formerly Column Families):** Similar to tables in relational databases, but with more flexibility. Rows within the same table do not need to share the exact same set of columns.
- **Rows:** Uniquely identified by a row key (primary key).
- **Columns:** Each column has a name, value, and timestamp.
- **Primary Key:** Composed of a partition key and optional clustering columns.
 - **Partition Key:** Determines how data is distributed across the cluster, ensuring that all data for a given partition key resides on specific nodes.
 - **Clustering Columns:** Define the order in which rows are stored within a partition, enabling efficient range queries.
- **Denormalization:** Cassandra emphasizes denormalization to optimize for query performance, often storing related data together in a single table to avoid joins.

Cassandra Examples

1. Logging System:

- **Query:** Retrieve the 10 most recent messages for a given source.
- **Table Design:** LogMessage_by_Source table with sourceName as the partition key and messageTime as a clustering column (sorted in descending order) to efficiently retrieve recent messages.

2. Time-Series Data:

- **Query:** Store and retrieve sensor readings over time.
- **Table Design:** A table with a compound primary key consisting of a sensor_id (partition key) and a timestamp (clustering column) for ordering the readings.

3. User Profile Data:

- **Query:** Retrieve a user's profile information.
- **Table Design:** A users table with user_id as the partition key to quickly access individual user profiles.

Cassandra Clients

Various client drivers and tools are available to interact with Cassandra:

- **Official Drivers:**

Apache Cassandra provides official drivers for popular languages like Java, Python, C#, Node.js, and C++. These drivers offer APIs to connect to Cassandra clusters, execute CQL (Cassandra Query Language) queries, and map data to application objects.

- **Object Mappers:**

Libraries like DataStax Object Mapper (for Java) simplify development by allowing developers to interact with Cassandra using object-oriented paradigms, reducing the need to write raw CQL queries.

- **CQLSH:**

A command-line shell provided with Cassandra, allowing users to execute CQL commands directly to interact with the database.

- **Third-Party Tools:**

Various third-party tools and GUIs exist for managing and interacting with Cassandra clusters, offering features like data browsing, query execution, and cluster monitoring.

Hadoop integration in big data analytics

Hadoop's integration into big data analytics is fundamental due to its ability to handle the storage and processing of massive datasets across distributed computing environments. It provides a robust, scalable, and cost-effective foundation for various analytical tasks.

Key aspects of Hadoop's integration in big data analytics:

- **Distributed Storage with HDFS (Hadoop Distributed File System):**
 - HDFS provides a fault-tolerant and highly scalable file system for storing petabytes of data across clusters of commodity hardware.
 - It ensures data availability and reliability by replicating data blocks across multiple nodes, making it ideal for large-scale data lakes and data warehousing for analytics.
- **Distributed Processing with MapReduce and other frameworks:**
 - **MapReduce:** This programming model enables parallel processing of large datasets across the HDFS cluster, breaking down complex tasks into smaller, manageable units.
 - **YARN (Yet Another Resource Negotiator):** YARN manages cluster resources and schedules processing jobs, allowing various processing frameworks (like Spark, Hive, Pig) to coexist and efficiently utilize the cluster.
 - **Apache Spark:** Often used in conjunction with Hadoop, Spark provides in-memory processing capabilities for faster analytics, including real-time processing, machine learning, and graph processing, leveraging HDFS for data storage.
 - **Apache Hive:** Enables SQL-like querying of data stored in HDFS, making it accessible to data analysts familiar with SQL.

- **Data Ingestion and Integration:**

- Hadoop integrates with various data sources and ingestion tools (e.g., Apache Sqoop for relational databases, Apache Flume for log data) to bring diverse data into HDFS for analysis.
- ETL (Extract, Transform, Load) processes can be performed within the Hadoop ecosystem using tools like Hive or Spark.

- **Analytical Tools and Applications:**

- Hadoop provides the underlying infrastructure for various analytical tools and applications, including:
 - **Machine Learning:** Frameworks like Apache Mahout and Spark MLlib can run on Hadoop to build and deploy machine learning models on large datasets.
 - **Data Warehousing:** Hadoop can serve as a data lake for storing raw and processed data, which can then be queried and analyzed by data warehousing solutions.
 - **Business Intelligence (BI):** Data processed in Hadoop can be integrated with BI tools like Power BI or Tableau for visualization and reporting.

- **Cloud Integration:**

- Hadoop ecosystems can be deployed on cloud platforms like AWS EMR and Azure HDInsight, offering managed services and scalability on demand.

- **Ecosystem Integration**

- a) Hive + Hadoop

- Hive converts SQL queries into MapReduce / Tez / Spark jobs.
 - Data is stored in HDFS.
 - Hive Metastore keeps table metadata.

- b) Pig + Hadoop

- Pig Latin scripts are converted into MapReduce jobs.
 - Used for ETL and data workflows.

- c) HBase + Hadoop

- HBase stores real-time data in a NoSQL model.
 - Actual files (HFiles) are stored in HDFS.

- Hadoop MapReduce or Spark can process HBase data.

In essence, Hadoop provides the foundational infrastructure for storing and processing the massive volumes of data required for big data analytics, enabling organizations to extract valuable insights and drive data-driven decision-making.

Introduction to Apache Pig

Pig Represents Big Data as data flows. Pig is a high-level platform or tool which is used to process the large datasets. It provides a high-level of abstraction for processing over the MapReduce. It provides a high-level scripting language, known as *Pig Latin* which is used to develop the data analysis codes. First, to process the data which is stored in the HDFS, the programmers will write the scripts using the Pig Latin Language. Internally *Pig Engine* (a component of Apache Pig) converted all these scripts into a specific map and reduce task. But these are not visible to the programmers in order to provide a high-level of abstraction. Pig Latin and Pig Engine are the two main components of the Apache Pig tool. The result of Pig always stored in the HDFS.

Note: Pig Engine has two type of execution environment i.e. a *local execution environment* in a single [JVM](#) (used when dataset is small in size) and *distributed execution environment* in a Hadoop Cluster.

Need of Pig: One limitation of MapReduce is that the development cycle is very long. Writing the reducer and mapper, compiling packaging the code, submitting the job and retrieving the output is a time-consuming task. Apache Pig reduces the time of development using the multi-query approach. Also, Pig is beneficial for programmers who are not from [Java](#) background. 200 lines of Java code can be written in only 10 lines using the Pig Latin language. Programmers who have SQL knowledge needed less effort to learn Pig Latin.

- It uses query approach which results in reducing the length of the code.
- Pig Latin is SQL like language.
- It provides many builtIn operators.
- It provides nested data types (tuples, bags, map).

Evolution of Pig: Earlier in 2006, Apache Pig was developed by Yahoo's researchers. At that time, the main idea to develop Pig was to execute the

MapReduce jobs on extremely large datasets. In the year 2007, it moved to Apache Software Foundation(ASF) which makes it an open source project. The first version(0.1) of Pig came in the year 2008. The latest version of Apache Pig is 0.18 which came in the year 2017.

Features of Apache Pig:

- For performing several operations Apache Pig provides rich sets of operators like the filtering, joining, sorting, aggregation etc.
- Easy to learn, read and write. Especially for SQL-programmer, Apache Pig is a boon.
- Apache Pig is extensible so that you can make your own process and user-defined functions(UDFs) written in python, java or other programming languages .
- Join operation is easy in Apache Pig.
- Fewer lines of code.
- Apache Pig allows splits in the pipeline.
- By integrating with other components of the Apache Hadoop ecosystem, such as Apache Hive, Apache Spark, and Apache ZooKeeper, Apache Pig enables users to take advantage of these components' capabilities while transforming data.
- The data structure is multivalued, nested, and richer.
- Pig can handle the analysis of both structured and unstructured data.

Difference between Pig and MapReduce

Apache Pig	MapReduce
It is a scripting language.	It is a compiled programming language.
Abstraction is at higher level.	Abstraction is at lower level.
It have less line of code as compared to MapReduce.	Lines of code is more.
Less effort is needed for Apache Pig.	More development efforts are required for MapReduce.
Code efficiency is less as compared to MapReduce.	As compared to Pig efficiency of code is higher.
Pig provides built in functions for ordering, sorting and union.	Hard to perform data operations.
It allows nested data types like map, tuple and bag	It does not allow nested data types

The language used to analyze data in Hadoop using Pig is known as **Pig Latin**. It is a highlevel data processing language which provides a rich set of data types and operators to perform various operations on the data.

To perform a particular task Programmers using Pig, programmers need to write a Pig script using the Pig Latin language, and execute them using any of the execution mechanisms (Grunt Shell, UDFs, Embedded). After execution, these scripts will go through a series of transformations applied by the Pig Framework, to produce the desired output.

Internally, Apache Pig converts these scripts into a series of MapReduce jobs, and thus, it makes the programmers job easy. The architecture of Apache Pig is shown below.

Types of Data Models in Apache Pig: It consist of the 4 types of data models as follows:

- **Atom:** It is a atomic data value which is used to store as a string. The main use of this model is that it can be used as a number and as well as a string.
- **Tuple:** It is an ordered set of the fields.
- **Bag:** It is a collection of the tuples.
- **Map:** It is a set of key/value pairs.

Applications of Apache Pig:

- For exploring large datasets Pig Scripting is used.
- Provides the supports across large data-sets for Ad-hoc queries.
- In the prototyping of large data-sets processing algorithms.
- Required to process the time sensitive data loads.
- For collecting large amounts of datasets in form of search logs and web crawls.
- Used where the analytical insights are needed using the sampling.

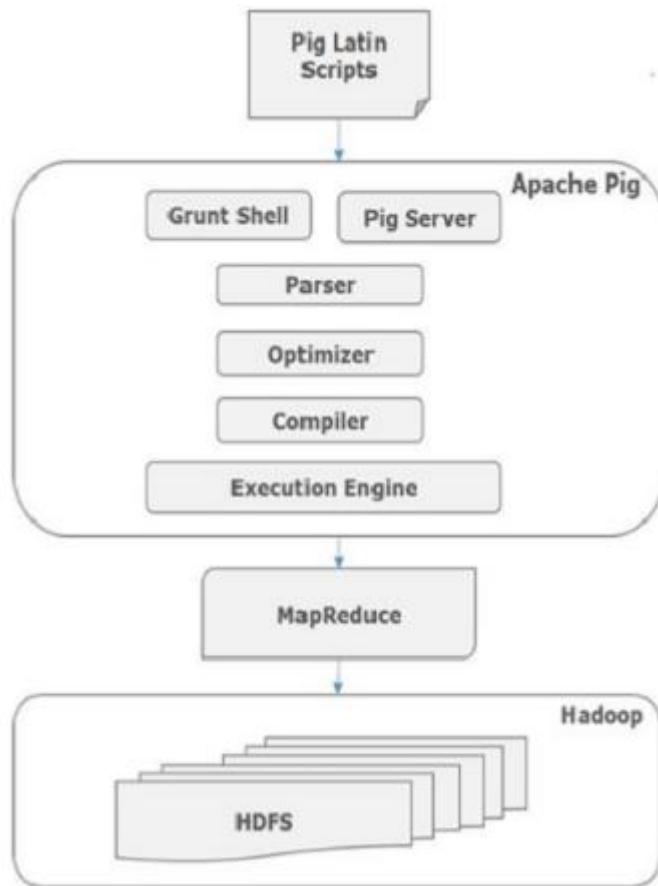


Fig:APACHE Pig architecture

Apache Pig Components

As shown in the figure, there are various components in the Apache Pig framework. Let us take a look at the major components.

Parser

Initially the Pig Scripts are handled by the Parser. It checks the syntax of the script, does type checking, and other miscellaneous checks. The output of the parser will be a DAG (directed acyclic graph), which represents the Pig Latin statements and logical operators.

In the DAG, the logical operators of the script are represented as the nodes and the data flows are represented as edges.

Optimizer

The logical plan (DAG) is passed to the logical optimizer, which carries out the logical optimizations such as projection and pushdown.

Compiler

The compiler compiles the optimized logical plan into a series of MapReduce jobs.

Execution engine

Finally the MapReduce jobs are submitted to Hadoop in a sorted order. Finally, these MapReduce jobs are executed on Hadoop producing the desired results.

Apache Pig Execution Modes

You can run Apache Pig in two modes, namely, **Local Mode** and **HDFS mode**.

Local Mode

In this mode, all the files are installed and run from your local host and local file system. There is no need of Hadoop or HDFS. This mode is generally used for testing purpose.

MapReduce Mode

MapReduce mode is where we load or process the data that exists in the Hadoop File System (HDFS) using Apache Pig. In this mode, whenever we execute the Pig Latin statements to process the data, a MapReduce job is invoked in the back-end to perform a particular operation on the data that exists in the HDFS.

Apache Pig Execution Mechanisms

Apache Pig scripts can be executed in three ways, namely, interactive mode, batch mode, and embedded mode.

- **Interactive Mode** (Grunt shell) – You can run Apache Pig in interactive mode using the Grunt shell. In this shell, you can enter the Pig Latin statements and get the output (using Dump operator).
- **Batch Mode** (Script) – You can run Apache Pig in Batch mode by writing the Pig Latin script in a single file with **.pig** extension.
- **Embedded Mode** (UDF) – Apache Pig provides the provision of defining our own functions (**User Defined Functions**) in programming languages such as Java, and using them in our script.

Invoking the Grunt Shell

You can invoke the Grunt shell in a desired mode (local/MapReduce) using the **-x** option as shown below.

Local mode	MapReduce mode
Command – \$./pig x local	Command – \$./pig -x mapreduce
Output – <pre>15/09/28 10:13:03 INFO pig.Main: Logging error messages to: /home/Hadoop/pig_1443415383991.log 2015-09-28 10:13:04,838 [main] INFO org.apache.pig.backend.hadoop.execution engine.HExecutionEngine - Connecting to hadoop file system at: file:/// grunt></pre>	Output – <pre>15/09/28 10:28:46 INFO pig.Main: Logging error messages to: /home/Hadoop/pig_1443416326123.log 2015-09-28 10:28:46,427 [main] INFO org.apache.pig.backend.hadoop.execution engine.HExecutionEngine - Connecting to hadoop file system at: file:/// grunt></pre>

Either of these commands gives you the Grunt shell prompt as shown below.

```
grunt>
```

You can exit the Grunt shell using **ctrl + d**.

After invoking the Grunt shell, you can execute a Pig script by directly entering the Pig Latin statements in it.

```
grunt> customers = LOAD 'customers.txt' USING PigStorage(',');
```

Executing Apache Pig in Batch Mode

You can write an entire Pig Latin script in a file and execute it using the **x command**. Let us suppose we have a Pig script in a file named **sample_script.pig** as shown below.

Sample_script.pig

```
student = LOAD 'hdfs://localhost:9000/pig_data/student.txt' USING
PigStorage(',') as (id:int,name:chararray,city:chararray);
```

```
Dump student;
```

Now, you can execute the script in the above file as shown below.

Local mode	MapReduce mode
\$ pig -x local Sample_script.pig	\$ pig -x mapreduce Sample_script.pig

Note – We will discuss in detail how to run a Pig script in **Bach mode** and in **embedded mode** in subsequent chapters.

After invoking the Grunt shell, you can run your Pig scripts in the shell. In addition to that, there are certain useful shell and utility commands provided by the Grunt shell. This chapter explains the shell and utility commands provided by the Grunt shell.

Grunt Shell Commands

The Grunt shell of Apache Pig is mainly used to write Pig Latin scripts. Prior to that, we can invoke any shell commands using **sh** and **fs**.

sh Command

Using **sh** command, we can invoke any shell commands from the Grunt shell. Using **sh** command from the Grunt shell, we cannot execute the commands that are a part of the shell environment (**ex** – cd).

Syntax

Given below is the syntax of **sh** command.

```
grunt> sh shell command parameters
```

Example

We can invoke the **ls** command of Linux shell from the Grunt shell using the **sh** option as shown below. In this example, it lists out the files in the **/pig/bin/** directory.

```
grunt> sh ls
```

```
pig
```

```
pig_1444799121955.log
```

```
pig.cmd
```

```
pig.py
```

fs Command

Using the **fs** command, we can invoke any FsShell commands from the Grunt shell.

Syntax

Given below is the syntax of **fs** command.

```
grunt> sh File System command parameters
```

Example

We can invoke the `ls` command of HDFS from the Grunt shell using `fs` command. In the following example, it lists the files in the HDFS root directory.

```
grunt> fs ls
```

Found 3 items

```
drwxrwxrwx - Hadoop supergroup      0 2015-09-08 14:13 Hbase
drwxr-xr-x  - Hadoop supergroup      0 2015-09-09 14:52 seqgen_data
drwxr-xr-x  - Hadoop supergroup      0 2015-09-08 11:30 twitter_data
```

In the same way, we can invoke all the other file system shell commands from the Grunt shell using the **fs** command.

Utility Commands

The Grunt shell provides a set of utility commands. These include utility commands such as **clear**, **help**, **history**, **quit**, and **set**; and commands such as **exec**, **kill**, and **run** to control Pig from the Grunt shell. Given below is the description of the utility commands provided by the Grunt shell.

clear Command

The **clear** command is used to clear the screen of the Grunt shell.

Syntax

You can clear the screen of the grunt shell using the **clear** command as shown below.

```
grunt> clear
```

help Command

The **help** command gives you a list of Pig commands or Pig properties.

Usage

You can get a list of Pig commands using the **help** command as shown below.

```
grunt> help
```

Commands: <pig latin statement>; - See the PigLatin manual for details:

<http://hadoop.apache.org/pig>

File system commands: fs <fs arguments> - Equivalent to Hadoop dfs command:

http://hadoop.apache.org/common/docs/current/hdfs_shell.html

Diagnostic Commands: describe <alias>[:<alias>] - Show the schema for the alias.

Utility Commands: exec [-param <param_name>=param_value] [-param_file <file_name>] <script> -

Execute the script with access to grunt environment including aliases.

history Command

This command displays a list of statements executed / used so far since the Grunt shell is invoked.

Usage

Assume we have executed three statements since opening the Grunt shell.

```
grunt> customers = LOAD 'hdfs://localhost:9000/pig_data/customers.txt'  
USING PigStorage(',');
```

```
grunt> orders = LOAD 'hdfs://localhost:9000/pig_data/orders.txt' USING  
PigStorage(',');
```

```
grunt> student = LOAD 'hdfs://localhost:9000/pig_data/student.txt' USING  
PigStorage(',');
```

Then, using the **history** command will produce the following output.

```
grunt> history
```

```
customers = LOAD 'hdfs://localhost:9000/pig_data/customers.txt' USING  
PigStorage(',');
```

```
orders = LOAD 'hdfs://localhost:9000/pig_data/orders.txt' USING  
PigStorage(',');
```

```
student = LOAD 'hdfs://localhost:9000/pig_data/student.txt' USING  
PigStorage(',');
```

set Command

The **set** command is used to show/assign values to keys used in Pig.

Usage

Using this command, you can set values to the following keys.

Key	Description and values
default_parallel	You can set the number of reducers for a map job by passing any whole number as a value to this key.
debug	You can turn off or turn on the debugging freature in Pig by passing on/off to this key.

job.name	You can set the Job name to the required job by passing a string value to this key.
job.priority	You can set the job priority to a job by passing one of the following values to this key – • very_low • low • normal • high • very_high
stream.skippath	For streaming, you can set the path from where the data is not to be transferred, by passing the desired path in the form of a string to this key.

quit Command

You can quit from the Grunt shell using this command.

Usage

Quit from the Grunt shell as shown below.

```
grunt> quit
```

Let us now take a look at the commands using which you can control Apache Pig from the Grunt shell.

exec Command

Using the **exec** command, we can execute Pig scripts from the Grunt shell.

Syntax

Given below is the syntax of the utility command **exec**.

```
grunt> exec [param param_name = param_value] [param_file file_name] [script]
```

Example

Let us assume there is a file named **student.txt** in the **/pig_data/** directory of HDFS with the following content.

Student.txt

001,Rajiv,Hyderabad

002,siddarth,Kolkata

003,Rajesh,Delhi

And, assume we have a script file named **sample_script.pig** in the **/pig_data/** directory of HDFS with the following content.

Sample_script.pig

```
student = LOAD 'hdfs://localhost:9000/pig_data/student.txt' USING
PigStorage(',')
as (id:int,name:chararray,city:chararray);
```

Dump student;

Now, let us execute the above script from the Grunt shell using the **exec** command as shown below.

```
grunt> exec /sample_script.pig
```

Output

The **exec** command executes the script in the **sample_script.pig**. As directed in the script, it loads the **student.txt** file into Pig and gives you the result of the Dump operator displaying the following content.

(1,Rajiv,Hyderabad)

(2,siddarth,Kolkata)

(3,Rajesh,Delhi)

kill Command

You can kill a job from the Grunt shell using this command.

Syntax

Given below is the syntax of the **kill** command.

```
grunt> kill JobId
```

Example

Suppose there is a running Pig job having id **Id_0055**, you can kill it from the Grunt shell using the **kill** command, as shown below.

```
grunt> kill Id_0055
```

run Command

You can run a Pig script from the Grunt shell using the **run** command

Syntax

Given below is the syntax of the **run** command.

```
grunt> run [param param_name = param_value] [param_file file_name] script
```

Example

Let us assume there is a file named **student.txt** in the **/pig_data/** directory of HDFS with the following content.

Student.txt

001,Rajiv,Hyderabad

002,siddarth,Kolkata

003,Rajesh,Delhi

And, assume we have a script file named **sample_script.pig** in the local filesystem with the following content.

Sample_script.pig

```
student = LOAD 'hdfs://localhost:9000/pig_data/student.txt' USING  
    PigStorage(',') as (id:int,name:chararray,city:chararray);
```

Now, let us run the above script from the Grunt shell using the run command as shown below.

```
grunt> run /sample_script.pig
```

You can see the output of the script using the **Dump operator** as shown below.

```
grunt> Dump;
```

(1,Rajiv,Hyderabad)

(2,siddarth,Kolkata)

(3,Rajesh,Delhi)

Note – The difference between **exec** and the **run** command is that if we use **run**, the statements from the script are available in the command history.

Pig Latin Data Model

As discussed in the previous chapters, the data model of Pig is fully nested. A **Relation** is the outermost structure of the Pig Latin data model. And it is a **bag** where –

- A bag is a collection of tuples.
- A tuple is an ordered set of fields.
- A field is a piece of data.

Pig Latin Statements

While processing data using Pig Latin, **statements** are the basic constructs.

- These statements work with **relations**. They include **expressions** and **schemas**.
- Every statement ends with a semicolon (;).
- We will perform various operations using operators provided by Pig Latin, through statements.
- Except LOAD and STORE, while performing all other operations, Pig Latin statements take a relation as input and produce another relation as output.
- As soon as you enter a **Load** statement in the Grunt shell, its semantic checking will be carried out. To see the contents of the schema, you need to use the **Dump** operator. Only after performing the **dump** operation, the MapReduce job for loading the data into the file system will be carried out.

Example

Given below is a Pig Latin statement, which loads data to Apache Pig.

```
grunt> Student_data = LOAD 'student_data.txt' USING PigStorage(',') as  
  ( id:int, firstname:chararray, lastname:chararray, phone:chararray,  
    city:chararray );
```


Pig Latin Data types

Given below table describes the Pig Latin data types.

S.N.	Data Type	Description & Example
1	int	Represents a signed 32-bit integer. Example : 8
2	long	Represents a signed 64-bit integer. Example : 5L
3	float	Represents a signed 32-bit floating point. Example : 5.5F
4	double	Represents a 64-bit floating point. Example : 10.5
5	chararray	Represents a character array (string) in Unicode UTF-8 format. Example : tutorials point
6	Bytearray	Represents a Byte array (blob).
7	Boolean	Represents a Boolean value. Example : true/ false.
8	Datetime	Represents a date-time. Example : 1970-01-01T00:00:00.000+00:00
9	Biginteger	Represents a Java BigInteger. Example : 60708090709

10	BigDecimal	Represents a Java BigDecimal Example : 185.98376256272893883
Complex Types		
11	Tuple	A tuple is an ordered set of fields. Example : (raja, 30)
12	Bag	A bag is a collection of tuples. Example : {(raju,30),(Mohhammad,45)}
13	Map	A Map is a set of key-value pairs. Example : [name#Raju, age#30]

Null Values

Values for all the above data types can be NULL. Apache Pig treats null values in a similar way as SQL does.

A null can be an unknown value or a non-existent value. It is used as a placeholder for optional values. These nulls can occur naturally or can be the result of an operation.

Pig Latin Arithmetic Operators

The following table describes the arithmetic operators of Pig Latin. Suppose a = 10 and b = 20.

Operator	Description	Example
+	Addition – Adds values on either side of the operator	a + b will give 30
–	Subtraction – Subtracts right hand operand from left hand operand	a – b will give –10

*	Multiplication – Multiplies values on either side of the operator	a * b will give 200
/	Division – Divides left hand operand by right hand operand	b / a will give 2
%	Modulus – Divides left hand operand by right hand operand and returns remainder	b % a will give 0
? :	Bincond – Evaluates the Boolean operators. It has three operands as shown below. variable x = (expression) ? value1 if true : value2 if false.	b = (a == 1)? 20: 30; if a = 1 the value of b is 20. if a!=1 the value of b is 30.
CASE WHEN THEN ELSE END	Case – The case operator is equivalent to nested bincond operator.	CASE f2 % 2 WHEN 0 THEN 'even' WHEN 1 THEN 'odd' END

Pig Latin Comparison Operators

The following table describes the comparison operators of Pig Latin.

Operator	Description	Example
==	Equal – Checks if the values of two operands are equal or not; if yes, then the condition becomes true.	(a = b) is not true
!=	Not Equal – Checks if the values of two operands are equal or not. If the values are not equal, then condition becomes true.	(a != b) is true.

>	Greater than – Checks if the value of the left operand is greater than the value of the right operand. If yes, then the condition becomes true.	(a > b) is not true.
<	Less than – Checks if the value of the left operand is less than the value of the right operand. If yes, then the condition becomes true.	(a < b) is true.
>=	Greater than or equal to – Checks if the value of the left operand is greater than or equal to the value of the right operand. If yes, then the condition becomes true.	(a >= b) is not true.
<=	Less than or equal to – Checks if the value of the left operand is less than or equal to the value of the right operand. If yes, then the condition becomes true.	(a <= b) is true.
matches	Pattern matching – Checks whether the string in the left-hand side matches with the constant in the right-hand side.	f1 matches '.*tutorial.*'

Pig Latin Type Construction Operators

The following table describes the Type construction operators of Pig Latin.

Operator	Description	Example
()	Tuple constructor operator – This operator is used to construct a tuple.	(Raju, 30)
{}	Bag constructor operator – This operator is used to construct a bag.	{(Raju, 30), (Mohammad, 45)}
[]	Map constructor operator – This operator is used to construct a tuple.	[name#Raja, age#30]

Pig Latin Relational Operations

The following table describes the relational operators of Pig Latin.

Operator	Description
Loading and Storing	
LOAD	To Load the data from the file system (local/HDFS) into a relation.
STORE	To save a relation to the file system (local/HDFS).
Filtering	
FILTER	To remove unwanted rows from a relation.
DISTINCT	To remove duplicate rows from a relation.
FOREACH, GENERATE	To generate data transformations based on columns of data.
STREAM	To transform a relation using an external program.
Grouping and Joining	
JOIN	To join two or more relations.
COGROUP	To group the data in two or more relations.
GROUP	To group the data in a single relation.
CROSS	To create the cross product of two or more relations.
Sorting	

ORDER	To arrange a relation in a sorted order based on one or more fields (ascending or descending).
LIMIT	To get a limited number of tuples from a relation.
Combining and Splitting	
UNION	To combine two or more relations into a single relation.
SPLIT	To split a single relation into two or more relations.
Diagnostic Operators	
DUMP	To print the contents of a relation on the console.
DESCRIBE	To describe the schema of a relation.
EXPLAIN	To view the logical, physical, or MapReduce execution plans to compute a relation.
ILLUSTRATE	To view the step-by-step execution of a series of statements.

Steps in Developing Pig Latin Scripts

a) Load Data

Use the LOAD operator to read data from HDFS or local file system.

```
students = LOAD 'hdfs:/data/students.txt'
```

```
USING PigStorage(',')
```

```
AS (id:int, name:chararray, dept:chararray, marks:int);
```

b) Data Transformation

Pig Latin provides many operators for transforming data.

1. FILTER

```
cse_students = FILTER students BY dept == 'CSE';
```

2. FOREACH – GENERATE

Used for column selection or computation.

```
names = FOREACH students GENERATE name, marks*1.1;
```

3. GROUP

Group records by a column.

```
grp = GROUP students BY dept;
```

4. JOIN

```
joined = JOIN students BY id, attendance BY id;
```

5. ORDER

```
ordered = ORDER students BY marks DESC;
```

6. DISTINCT

```
unique_depts = DISTINCT students.dept;
```

c) Store / Dump Results

STORE (write to HDFS/local)

```
STORE cse_students INTO '/output/cse' USING PigStorage(',');
```

DUMP (print to screen)

```
DUMP cse_students;
```

Complete Pig Latin Example Program

Task: Find average marks department-wise.

```
students = LOAD '/data/students.txt'
```

```
USING PigStorage(',')
```

```
AS (id:int, name:chararray, dept:chararray, marks:int);
```

```
grouped = GROUP students BY dept;
```

```
avg_marks = FOREACH grouped
```

```
    GENERATE group AS dept,
```

```
        AVG(students.marks) AS avg_mark;
```

```
STORE avg_marks INTO '/output/avg' USING PigStorage(',');
```

3. Pig Latin Features Useful for Development

a) User-Defined Functions (UDFs)

You can write custom Java functions when built-in functions are not enough.

```
REGISTER 'myudfs.jar';
```

```
result = FOREACH students GENERATE myudf(name);
```

b) Execution Modes

- **Local Mode** – runs on single machine
- **MapReduce Mode** – runs on HDFS cluster

c) Schema-on-Read

Schemas are given at load time; files do not require predefined formats.

4. Advantages of Developing in Pig Latin

- Simple, readable scripting language
- Automatically converts scripts into optimized MapReduce jobs
- Handles complex ETL (Extract–Transform–Load) pipelines
- Extensible via UDFs
- Faster development compared to writing MapReduce manually

5. Common Use Cases

- Log processing
- Clickstream analysis
- Data cleaning & transformation
- ETL pipelines in data lakes

Testing Pig Latin scripts, which are used in Apache Pig for data processing, involves several methods to ensure correctness and efficiency.

Testing Pig Latin Scripts

1. Interactive Testing with Grunt Shell and DUMP:

- The Grunt shell provides an interactive environment to execute Pig Latin commands.
- The DUMP operator can be used within Grunt to immediately execute a Pig Latin statement and display its results to the screen. This is useful for prototyping and debugging individual steps of a script, verifying that the expected output is generated. However, DUMP should generally be avoided in production scripts as it can hinder multi-query optimizations.

2. Script Execution in Batch Mode:

- Pig Latin statements can be written in a .pig file and executed in batch mode from the command line. This allows for running complete scripts and observing their behavior with larger datasets.

3. Debugging Operators:

- DESCRIBE:

This operator helps in understanding the schema of a relation, ensuring data types and field names are as expected.

- EXPLAIN:

This operator provides insights into the execution plan (logical, physical, and MapReduce) that Pig will generate for a given relation, aiding in performance optimization and understanding how the script will be processed.

- ILLUSTRATE:

This operator allows for a step-by-step visualization of the execution of a series of statements, showing the state of the data at each stage.

4. Unit Testing with PigUnit:

- For more rigorous and automated testing, especially in development and continuous integration environments, the Apache Group provides the PigUnit library.
- PigUnit enables unit testing of Pig scripts using JUnit, allowing developers to test their logic, UDFs (User Defined Functions), and ensure

compatibility when changes are made to scripts or the underlying Pig/Hadoop versions.

- PigUnit can run in either Local mode (which uses the local file system and doesn't require a Hadoop cluster) or MapReduce mode (which requires a Hadoop cluster and HDFS installation). Local mode is typically used for development testing.

5. Code Coverage with Clover (for Pig's own development):

- For those developing Pig itself, code coverage of unit tests can be measured using Clover, a tool for measuring code coverage. This helps ensure that the tests adequately cover the codebase.

Hive – Data Types, File Formats

Apache Hive supports various data types and file formats for storing and managing data in a data warehousing context built on Hadoop.

Hive Data Types:

Hive data types are categorized into primitive and complex types.

- **Primitive Data Types:**
 - **Numeric Types:** TINYINT, SMALLINT, INT, BIGINT (for integers of various sizes), FLOAT, DOUBLE, DECIMAL (for floating-point and arbitrary-precision numbers).
 - **Date/Time Types:** TIMESTAMP (for date and time with millisecond precision), DATE (for date only), INTERVAL.
 - **String Types:** STRING (for variable-length character strings), VARCHAR (for variable-length character strings with a specified maximum length), CHAR (for fixed-length character strings).
 - **Miscellaneous Types:** BOOLEAN (for true/false values), BINARY (for byte arrays).
- **Complex Data Types:**
 - **ARRAY:** An ordered collection of elements of the same data type.

Code

ARRAY<data_type>

- **MAP:** A collection of key-value pairs, where keys are of one data type and values are of another.

Code

MAP<key_data_type, value_data_type>

- **STRUCT:** A record type that contains named fields of potentially different data types.

Code

STRUCT<field_name1:data_type1, field_name2:data_type2, ...>

- **UNIONTYPE:** A type that can hold a value of any one of a specified set of data types.

Hive File Formats:

Hive supports several file formats that determine how data is stored on HDFS, impacting performance and storage efficiency.

- **TextFile:**
 - The default format. Data is stored as plain text, with records typically separated by newlines and fields by delimiters.
 - Simple and human-readable, but less efficient for storage and query performance than binary formats.
- **SequenceFile:**
 - A binary, key-value pair format in Hadoop.
 - Supports compression and is splittable, offering better performance than TextFile, especially for large datasets.
- **RCFile (Record Columnar File):**
 - A columnar storage format that stores data row-wise within blocks and then column-wise within those blocks.
 - Optimized for read-heavy workloads and offers good compression.
- **ORCFile (Optimized Row Columnar File):**
 - An advanced columnar storage format designed for Hive.

- Offers superior compression, predicate pushdown, and vectorized reads for enhanced query performance.
- **Parquet:**
 - Another widely used columnar storage format, not specific to Hive but well-integrated.
 - Provides efficient data compression and encoding schemes, and is highly compatible with various data processing frameworks.
- **Avro:**
 - A row-based data serialization system that provides rich data structures and a compact, fast binary data format.
 - Supports schema evolution, making it suitable for data interchange.

HIVEQL – DATA DEFINITION, DATA MANIPULATION & QUERIES

HiveQL (Hive Query Language) is an SQL-like language used in **Apache Hive** for querying and managing large datasets stored in **HDFS**. HiveQL queries are internally converted into **MapReduce / Tez / Spark** jobs for execution.

1. Data Definition Language (DDL)

DDL commands define databases, tables, and schemas in Hive.

a) Create Database

```
CREATE DATABASE college;
```

b) Drop Database

```
DROP DATABASE college CASCADE;
```

c) Create Table

```
CREATE TABLE students (
  id INT,
  name STRING,
  dept STRING,
  marks INT
)
```

ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';

d) Create External Table

(Hive does not own the data)

```
CREATE EXTERNAL TABLE logs (  
  userid STRING,  
  action STRING,  
  ts BIGINT  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY '\t'  
LOCATION '/data/logs';
```

e) Partitioned Table

```
CREATE TABLE sales(  
  item STRING,  
  amount INT  
)  
PARTITIONED BY (year INT, month INT)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ',';
```

f) Alter Table

Add Column

```
ALTER TABLE students ADD COLUMNS (email STRING);
```

Rename Table

```
ALTER TABLE students RENAME TO stu;
```

g) Drop Table

```
DROP TABLE students;
```

2. Data Manipulation Language (DML)

DML commands insert, load, or modify data.

a) Load Data into Table

```
LOAD DATA LOCAL INPATH '/home/user/students.csv'  
INTO TABLE students;
```

b) Insert Data

```
INSERT INTO TABLE students VALUES  
(101, 'Sam', 'CSE', 85);
```

Insert into partition

```
INSERT INTO TABLE sales PARTITION (year=2025, month=12)  
VALUES ('Laptop', 55000);
```

c) Insert-Overwrite

(Replaces existing data)

```
INSERT OVERWRITE TABLE topper  
SELECT * FROM students WHERE marks > 90;
```

d) Update Data

(Only for transactional tables)

```
UPDATE students  
SET marks = 90  
WHERE id = 101;
```

e) Delete Data

```
DELETE FROM students  
WHERE dept = 'ECE';
```

3. HiveQL Queries (SELECT Operations)

a) Simple Select

```
SELECT name, marks FROM students;
```

b) WHERE Clause

```
SELECT * FROM students WHERE dept = 'CSE';
```

c) DISTINCT

```
SELECT DISTINCT dept FROM students;
```

d) ORDER BY / SORT BY

```
SELECT * FROM students ORDER BY marks DESC;
```

e) GROUP BY + Aggregate Functions

```
SELECT dept, AVG(marks) AS avg_marks  
FROM students  
GROUP BY dept;
```

f) HAVING

```
SELECT dept, AVG(marks) AS avg_marks  
FROM students  
GROUP BY dept  
HAVING avg_marks > 75;
```

g) JOIN

```
SELECT s.name, m.subject, m.marks  
FROM students s  
JOIN marks m  
ON s.id = m.id;
```

h) LIMIT

```
SELECT * FROM students LIMIT 5;
```

i) Partition-Based Query

```
SELECT * FROM sales  
WHERE year = 2025 AND month = 12;
```

j) Built-in Functions

String:

```
SELECT UPPER(name) FROM students;
```

Date:

```
SELECT YEAR(date_col) FROM logs;
```

- **4. Summary Table**

Category	Examples
DDL	CREATE, ALTER, DROP, PARTITION
DML	LOAD, INSERT, UPDATE, DELETE
Queries	SELECT, JOIN, GROUP BY, ORDER BY, FUNCTIONS

For More Notes on HIVEQ1 refer the following text book

https://sharathkumarblog.wordpress.com/wp-content/uploads/2016/05/programming_hive.pdf for your reference